ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

LAB 01

Môn: Toán cho Trí tuệ nhân tạo

NGUYỄN MINH NHỰT

MSHV: 25C15019

Ngành: Trí tuệ nhân tạo

MẠNG NƠ-RON NHIỀU LỚP VÀ CƠ CHẾ HỌC

1. Mô tả mô hình

Bài toán đặt ra là phân loại dataset Iris gồm 150 mẫu, mỗi mẫu là một vector $x \in \mathbb{R}^4$ gồm 4 feature liên tục (sepal length, sepal width, petal length, petal width), thuộc một trong 3 lớp: Setosa, Versicolor, Virginica. Đây là bài toán phân loại đa lớp, output là phân bố xác suất trên 3 lớp. Mã nguồn bài toán có thể tìm thấy ở

Để giải quyết bài toán này, ta dùng MLP (Multilayer Perceptron) - kiến trúc gồm các layer xếp chồng, mỗi layer thực hiện phép biến đổi affine rồi áp hàm kích hoạt phi tuyến f. Với đủ nơ-ron, MLP có thể xấp xỉ bất kỳ hàm liên tục nào trên miền compact (universal approximation theorem). Mạng được thiết kế với 3 layer: input 4 chiều, hidden 8 nơ-ron, output 3 lớp.

2. Chương trình nguồn mở: PyTorch

PyTorch (Meta AI, mã nguồn mở tại github.com/pytorch/pytorch) xây dựng **dynamic computation graph** tại runtime: mỗi phép toán trên tensor có requires_grad=True đều được ghi vào đồ thị. Khi gọi loss.backward(), autograd duyệt ngược đồ thị và tích lũy gradient vào .grad của từng tham số. Bước cập nhật tham số được thực hiện bởi optimizer.step().

```
output = model(x)
loss   = criterion(output, y)
optimizer.zero_grad()   # xóa gradient cũ
loss.backward()         # tính gradient qua autograd
optimizer.step()        # W ← W - η·∇W
```

Cơ chế này tương ứng trực tiếp với backpropagation dẫn xuất ở Mục 5 và gradient descent ở Mục 6.

3. Kiến trúc và tham số

$$\text{Input } (4) \xrightarrow{W^{(1)}, b^{(1)}} \text{Hidden } (8) \xrightarrow{W^{(2)}, b^{(2)}} \text{Output } (3)$$

Mạng có 4 nhóm tham số cần học. $W^{(1)} \in \mathbb{R}^{8\times 4}$ và $b^{(1)} \in \mathbb{R}^8$ là weight và bias của hidden layer - biến đổi input 4 chiều thành vector 8 chiều. $W^{(2)} \in \mathbb{R}^{3\times 8}$ và $b^{(2)} \in \mathbb{R}^3$ là weight và bias của output layer - chiếu vector 8 chiều xuống 3 lớp phân loại. Tổng số tham số là $(8 \times 4 + 8) + (3 \times 8 + 3) = 40 + 27 = 67$.

4. Forward Pass

Trước khi trình bày công thức, ta định nghĩa các ký hiệu được sử dụng xuyên suốt. $x \in \mathbb{R}^4$ là vector input một mẫu. $z^{(1)} \in \mathbb{R}^8$ và $z^{(2)} \in \mathbb{R}^3$ là các vector pre-

activation (kết quả biến đổi affine trước khi qua hàm kích hoạt) tại hidden layer và output layer. $h \in \mathbb{R}^8$ là vector post-activation tại hidden layer. $\hat{y} \in \mathbb{R}^3$ là vector xác suất dự đoán tại output, với $\hat{y}_i$ là xác suất mẫu thuộc lớp $i \in \{1,2,3\}$. $L \in \mathbb{R}$ là giá trị loss đo sai lệch giữa dự đoán và nhãn thật. $f: \mathbb{R} \to \mathbb{R}$ là hàm kích hoạt phi tuyến, áp elementwise lên từng phần tử của vector, với f' là ký hiệu đạo hàm của f theo đối số của nó.

Forward pass tính lần lượt qua hai layer:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \qquad h = f(z^{(1)})$$

$$z^{(2)} = W^{(2)}h + b^{(2)}, \qquad \hat{y} = \mathrm{softmax}(z^{(2)})$$

Hàm softmax chuyển logit $z^{(2)}$ thành phân bố xác suất, đảm bảo $\hat{y}_i \geq 0$ và $\sum_i \hat{y}_i = 1$:

$$\boldsymbol{\hat{y}_i = \frac{e^{z_i^{(2)}}}{\sum_{k=1}^{3} e^{z_k^{(2)}}}, \quad i = 1, 2, 3}$$

5. Loss Function

Ta dùng categorical cross-entropy. Nhãn thật $y \in \mathbb{R}^3$ được biểu diễn dạng one-hot - tức $y_i = 1$ nếu mẫu thuộc lớp i, còn lại bằng 0. Loss đo khoảng cách giữa phân bố dự đoán $\hat{y}$ và nhãn thật y:

$$L = -\sum_{i=1}^{3} y_i \log \hat{y}_i$$

Vì y là one-hot, tổng này thực ra chỉ còn một số hạng duy nhất tương ứng với lớp đúng: $L = -\log \hat{y}_{i^*}$ với i^* là nhãn thật. Loss nhỏ khi mô hình gán xác suất cao cho đúng lớp.

6. Backpropagation

Mục tiêu là tính $\frac{\partial L}{\partial W^{(l)}}$ và $\frac{\partial L}{\partial b^{(l)}}$ cho $l = 1,2$ để cập nhật tham số. Ta áp dụng chain rule dọc theo luồng tính toán:

$$x \xrightarrow{W^{(1)}, b^{(1)}} z^{(1)} \xrightarrow{f} h \xrightarrow{W^{(2)}, b^{(2)}} z^{(2)} \xrightarrow{\mathrm{softmax}} \hat{y} \xrightarrow{\mathrm{CE}} L$$

Đặt $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$ là gradient của loss theo pre-activation tại layer l. Đây là đại lượng trung gian quan trọng: gradient theo $W^{(l)}$ và $b^{(l)}$ được dẫn xuất trực tiếp từ $\delta^{(l)}$ thông qua cấu trúc affine $z^{(l)} = W^{(l)}(\cdot) + b^{(l)}$.

6.1 Output layer

Tính $\delta^{(2)}$. Áp dụng chain rule qua softmax:

$$\frac{\partial L}{\partial z_i^{(2)}} = \sum_j \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_i^{(2)}}$$

Hai thành phần, trong đó $j \in \{1,2,3\}$ là chỉ số chạy qua tất cả các lớp output: $\frac{\partial L}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}$ từ cross-entropy, và đạo hàm softmax có hai trường hợp do tính normalize toàn bộ vector:

$$\frac{\partial \hat{y}_j}{\partial z_i^{(2)}} = \begin{cases} \hat{y}_i(1-\hat{y}_i) & j = i \\ -\hat{y}_j\hat{y}_i & j \neq i \end{cases}$$

Tách tổng thành số hạng $j = i$ và phần còn lại $j \neq i$:

$$\text{số hạng } j = i: \quad -\frac{y_i}{\hat{y}_i} \cdot \hat{y}_i(1-\hat{y}_i) = -y_i(1-\hat{y}_i)$$

$$\text{các số hạng } j \neq i: \quad \sum_{j \neq i} \left(-\frac{y_j}{\hat{y}_j}\right)(-\hat{y}_j\hat{y}_i) = \hat{y}_i \sum_{j \neq i} y_j$$

Cộng lại và dùng $\sum_j y_j = 1$ (vì y là one-hot):

$$\frac{\partial L}{\partial z_i^{(2)}} = -y_i(1-\hat{y}_i) + \hat{y}_i \sum_{j \neq i} y_j = -y_i + \hat{y}_i \underbrace{\sum_j y_j}_{=1} \Longrightarrow \boxed{\delta^{(2)} = \hat{y} - y}$$

Gradient theo $W^{(2)}$ và $b^{(2)}$. Vì $z^{(2)} = W^{(2)}h + b^{(2)}$, ta có $\frac{\partial z_i^{(2)}}{\partial W_{ij}^{(2)}} = h_j$ và $\frac{\partial z_i^{(2)}}{\partial b_i^{(2)}} = 1$, do đó:

$$\frac{\partial L}{\partial W_{ij}^{(2)}} = \delta_i^{(2)} \cdot h_j \Longrightarrow \boxed{\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}h^T, \qquad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}}$$

6.2 Hidden layer

Tính $\delta^{(1)}$. h ảnh hưởng đến L chỉ qua $z^{(2)} = W^{(2)}h + b^{(2)}$, nên $\frac{\partial z_i^{(2)}}{\partial h_j} = W_{ij}^{(2)}$ và:

$$\frac{\partial L}{\partial h_j} = \sum_i \frac{\partial L}{\partial z_i^{(2)}} \cdot W_{ij}^{(2)} \Longrightarrow \frac{\partial L}{\partial h} = W^{(2)T}\delta^{(2)}$$

Tiếp tục lan truyền qua $h = f(z^{(1)})$. Vì f áp elementwise, $\frac{\partial h_j}{\partial z_k^{(1)}} = 0$ khi $k \neq j$, nên:

$$\frac{\partial L}{\partial z_j^{(1)}} = \frac{\partial L}{\partial h_j} \cdot f'(z_j^{(1)}) \Longrightarrow \boxed{\delta^{(1)} = \left(W^{(2)T}\delta^{(2)}\right) \odot f'(z^{(1)})}$$

trong đó $\odot$ là Hadamard product (nhân elementwise).

Gradient theo $W^{(1)}$ và $b^{(1)}$. Cùng cấu trúc với output layer, vì $z^{(1)} = W^{(1)}x + b^{(1)}$:

$$\boxed{\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)}x^T, \qquad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}}$$

---

7. Gradient Descent

Sau khi có đủ gradient, tham số được cập nhật đồng thời cho cả hai layer:

$$W^{(l)} \leftarrow W^{(l)} - \eta\,\frac{\partial L}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \eta\,\frac{\partial L}{\partial b^{(l)}}, \quad l = 1,2$$

trong đó $\eta > 0$ là learning rate - kiểm soát độ lớn mỗi bước cập nhật. Trong PyTorch, optimizer.step() thực hiện bước này sau khi loss.backward() đã tính đầy đủ gradient.

8. Thực nghiệm: PyTorch vs Sklearn MLP

Dataset. Iris gồm 150 mẫu, 4 feature, 3 lớp. Chia 80/20 train/test, chuẩn hóa bằng StandardScaler để các feature có mean 0, std 1.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris  = load_iris()
X     = StandardScaler().fit_transform(iris.data)
y     = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
ate=42)
```

PyTorch MLP - SGD, lr=0.1, 200 epochs, cấu trúc Linear(4→8) + ReLU + Linear(8→3):

```
import torch, torch.nn as nn, torch.optim as optim

X_tr = torch.FloatTensor(X_train);  y_tr = torch.LongTensor(y_train)
```

```
X_te = torch.FloatTensor(X_test);   y_te = torch.LongTensor(y_test)

model   = nn.Sequential(nn.Linear(4,8), nn.ReLU(), nn.Linear(8,3))
opt     = optim.SGD(model.parameters(), lr=0.1)
loss_fn = nn.CrossEntropyLoss()

for _ in range(200):
    opt.zero_grad()
    loss_fn(model(X_tr), y_tr).backward()
    opt.step()

with torch.no_grad():
    acc = (model(X_te).argmax(1) == y_te).float().mean().item()
print(f"PyTorch accuracy: {acc:.4f}")
```

Sklearn MLP - cùng kiến trúc và hyperparameter:

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

clf = MLPClassifier(hidden_layer_sizes=(8,), activation='relu',
                    solver='sgd', learning_rate_init=0.1,
                    max_iter=200, random_state=42)
clf.fit(X_train, y_train)
print(f"Sklearn accuracy: {accuracy_score(y_test, clf.predict(X_test)):.4f}")
```

Cả hai đạt ~0.97 accuracy trên test set. Kết quả tương đương xác nhận tính đúng đắn của phần lý thuyết - hai implementation độc lập cùng hội tụ về cùng một nghiệm khi dùng cùng kiến trúc và optimizer.

PHƯƠNG PHÁP PARABOL THAY THẾ GRADIENT

1. Ý tưởng

Parabolic interpolation là kỹ thuật tối ưu **không dùng đạo hàm**: thay vì tính ∇L, phương pháp xấp xỉ L cục bộ bằng một parabol qua 3 điểm rồi nhảy thẳng đến cực tiểu của parabol đó.

Để tối ưu toàn bộ mạng, ta áp dụng **tối ưu tọa độ** (coordinate descent): lần lượt tối ưu từng tham số θ_i trong khi giữ nguyên các tham số còn lại. Với mỗi θ_i, ta đánh giá L tại ba điểm $\theta_i - \delta,\ \theta_i,\ \theta_i + \delta$ (với δ là bước dò nhỏ), nội suy parabol và cập nhật θ_i về cực tiểu của parabol. **Không cần backpropagation.**

2. Công thức

Cho 3 điểm $(x_1, L_1),\ (x_2, L_2),\ (x_3, L_3)$ với $x_1 < x_2 < x_3$, cực tiểu của parabol nội suy là:

$$x^* = x_2 - \frac{1}{2} \cdot \frac{(x_2 - x_1)^2 (L_2 - L_3) - (x_2 - x_3)^2 (L_2 - L_1)}{(x_2 - x_1)(L_2 - L_3) - (x_2 - x_3)(L_2 - L_1)}$$

Với $x_1 = \theta_i - \delta,\ x_2 = \theta_i,\ x_3 = \theta_i + \delta$, công thức rút gọn thành:

$$\theta_i^* = \theta_i - \frac{\delta\,(L_1 - L_3)}{2\,(2L_2 - L_1 - L_3)}$$

Điều kiện hợp lệ: mẫu số khác 0 và $2L_2 - L_1 - L_3 < 0$ (parabol mở lên, có cực tiểu). Nếu không thỏa, giữ nguyên θ_i.

Một vấn đề thực tế: khi loss thay đổi rất ít theo θ_i (mẫu số rất nhỏ), θ_i^* có thể bay rất xa. Ta giới hạn bước nhảy tối đa $|\theta_i^* - \theta_i| \le C\delta$ với $C = 5$:

$$\theta_i \leftarrow \text{clip}(\theta_i^*,\ \theta_i - C\delta,\ \theta_i + C\delta)$$

3. Áp dụng

Tại mỗi epoch, với mỗi tham số $\theta_i \in \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}\}$:

1. Tính $L_- = L(\dots, \theta_i - \delta, \dots)$, $L_0 = L(\dots, \theta_i, \dots)$, $L_+ = L(\dots, \theta_i + \delta, \dots)$
2. Nội suy parabol qua $(\theta_i - \delta, L_-)$, (θ_i, L_0), $(\theta_i + \delta, L_+)$
3. Cập nhật $\theta_i \leftarrow \theta_i^*$

Mạng có 67 tham số, mỗi epoch cần $2 \times 67 = 134$ lần tính forward pass — không cần tính gradient.

4. Implement (NumPy)

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris  = load_iris()
X     = StandardScaler().fit_transform(iris.data)
y     = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st
ate=42)

def one_hot(labels, k=3):
    Y = np.zeros((len(labels), k))
    Y[np.arange(len(labels)), labels] = 1
    return Y

Y_train = one_hot(y_train)

np.random.seed(42)
W1 = np.random.randn(8, 4) * 0.1;  b1 = np.zeros((8, 1))
W2 = np.random.randn(3, 8) * 0.1;  b2 = np.zeros((3, 1))

def relu(z):    return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max(axis=0, keepdims=True))
    return e / e.sum(axis=0, keepdims=True)

def forward(X, W1, b1, W2, b2):
    H    = relu(W1 @ X.T + b1)
```

```python
    Yhat = softmax(W2 @ H + b2)
    return Yhat

def cross_entropy(Yhat, Y):
    return -np.mean(np.sum(Y.T * np.log(Yhat + 1e-9), axis=0))

def parabolic_update(x0, L_minus, L0, L_plus, delta, C=5):
    """Return parabola minimum; fall back to x0 if degenerate.
    Clamps result to [x0 - C*delta, x0 + C*delta] to prevent runaway jumps."""
    denom = 2 * (2*L0 - L_minus - L_plus)
    if abs(denom) < 1e-10 or denom > 0:
        return x0
    x_star = x0 - delta * (L_minus - L_plus) / denom
    return float(np.clip(x_star, x0 - C*delta, x0 + C*delta))

delta = 0.1

for epoch in range(500):
    for matrix in [W1, b1, W2, b2]:
        for idx in np.ndindex(matrix.shape):
            orig = matrix[idx]
            L0      = cross_entropy(forward(X_train, W1, b1, W2, b2), Y_train)

            matrix[idx] = orig - delta
            L_minus = cross_entropy(forward(X_train, W1, b1, W2, b2), Y_train)

            matrix[idx] = orig + delta
            L_plus  = cross_entropy(forward(X_train, W1, b1, W2, b2), Y_train)

            matrix[idx] = parabolic_update(orig, L_minus, L0, L_plus, delta)

Yhat_te = forward(X_test, W1, b1, W2, b2)
acc = (Yhat_te.argmax(axis=0) == y_test).mean()
print(f"Parabolic MLP accuracy: {acc:.4f}")
```

5. Kết quả

PyTorch MLP (gradient descent) đạt 0.9467 ± 0.0340 trên 5-fold CV. Parabolic coordinate descent đạt 0.9267 ± 0.0490 — accuracy gần tương đương dù **không dùng đạo hàm lần nào**. Đánh đổi là mỗi epoch cần $2 \times 67 = 134$ lần forward pass thay vì 1 lần forward + 1 lần backward như gradient descent, và cần nhiều epoch hơn (500 so với 200) để hội tụ.